BEGINNER'S GUIDE · CONCEPTS EXPLAINED FROM ZERO

Understanding the Solar IoT Digital Twin and its ANN + Fuzzy brain

No prior knowledge assumed. Every idea is explained with an everyday analogy, then with the real numbers from this project, step by step.

How this guide is organised

PART A — **The physical world**
1 The story · 2 Digital twin · 3 Sun & battery · 4 Brain & task · 5 WiFi · 6 Server

PART B — **How a decision is made**
7 The bill (cost) · 8 One real minute, step by step · 9 Closed loop

PART C — **The intelligent layer**
10 Why AI · 11 ANN · 12 Fuzzy logic · 13 ANFIS · 14 Hybrid

PART D — **Proving it works**
15 Fair testing · 16 Results in plain words

PART E — **Explaining it yourself**
17 Common confusions · 18 A 2-minute script · 19 Self-test

Companion to Project_Guide.pdf (the technical reference). Read this one first.

1 The story behind the project

Imagine a small sensor box fixed on a machine in a factory, or on a pole in a farm field. Inside it is a tiny computer called an **ESP32**. It has no power cable. It lives on a **small solar panel** and a **rechargeable battery**, like a solar garden lamp.

Every minute, the box collects sensor data (vibration, temperature, a small camera image…) and must **process** it: clean the signal, find patterns, check for something abnormal. We call each of these jobs a **task**.

For every task, the box has exactly **two choices**:

Choice	What it means	Good side	Bad side
LOCAL	Do the task itself, on its own tiny processor	No WiFi needed; no waiting on anyone	Slow for big tasks; the processor uses battery while working
OFFLOAD	Send the data over WiFi to a powerful nearby computer (the **edge server**), and get the answer back	The server is about 12× faster	Sending data costs battery; a weak signal means repeating; the server may be busy

Everyday analogy
You have homework every hour. You can solve it yourself with pen and paper (LOCAL): slow, and it tires you. Or you can phone a friend who has a calculator (OFFLOAD): fast, but the call costs phone credit. If the network is bad you must repeat yourself, so the call takes longer and costs more. If your friend is busy, you wait on the line. And when your phone credit is almost finished, every call feels expensive. **Which is better depends on the moment.** That is exactly the decision this project studies.

The question the project answers: for each task, every minute, which choice is better, and why? It also asks how a real device could make that choice **intelligently** when it does not know everything in advance.

Remember this
LOCAL = do it yourself. OFFLOAD = send it to the server. Neither is always better; it depends on battery, signal, server and task size, all of which keep changing.

2 What is a "digital twin"?

Testing ideas on real hardware is slow and costly: you would need the device, a solar panel, weeks of weather, a WiFi network and a server, and you could not repeat the same day twice.

A **digital twin** is a **software copy** of the real system that behaves the same way, because it follows the same physical rules (equations). You can run a whole week in a few seconds, repeat it exactly, and try "what if" changes safely.

Everyday analogy
Pilots train in a **flight simulator** before flying a real plane. The simulator follows the physics of flight, so what you learn there works in the real plane. Our digital twin is a "flight simulator" for a solar IoT device.

In this project the twin is the Python file `digital_twin_simulator.py`. Each time it takes **one step**, one minute passes in the simulated world. It ran 7 days = 10,080 minutes = **10,080**

tasks, and saved every minute as one row of a table (`simulation_dataset.csv`).

In simple words
A digital twin is a realistic computer copy of a device. It lets us watch, test and learn without the real hardware.

3 Sun and battery: where the energy comes from

3.1 The solar panel

Sunlight strength is called **irradiance** (watts per square metre). At night it is zero. It rises after 6 am, peaks at noon and falls to zero at 6 pm, like a hill. Clouds cut it down. Each day in the twin is **Sunny**, **Partly Cloudy** or **Overcast**, and tomorrow's weather depends a little on today's, as in real life.

The panel turns sunlight into electricity. Ours is a small **1 watt** panel (about the size of a phone), and it loses about 15 % to heat and dust. So at noon on a perfect day it gives about 0.85 W; on an overcast day much less.

3.2 The battery

Everyday analogy
Think of the battery as a **water tank**. The solar panel is a tap filling it. The device is a hole draining it. **SOC (state of charge)** is how full the tank is, in %. During the day the tap is strong, so the tank fills; at night only the drain works, so the tank empties.

The device always uses about **0.14 W** just to stay alive (sensors on, WiFi connected). That is the "base load". On top of that, each task uses a little extra energy, which depends on whether it ran LOCAL or OFFLOAD. Our battery holds **12 watt-hours**. Charging and discharging each lose about 5 %, just like a real lithium battery.

In the 7-day dataset the battery moves between about 15 % and 57 %. It never sits full, so energy really matters for the decisions.

Remember this
Sun → panel → battery (SOC %). The device drains the battery all the time, and each task drains a bit more. Low SOC = energy is precious.

4 The device's brain and its tasks

4.1 How big is a task?

Every task has two properties:

- **Size**: how much data was collected, in kilobytes (KB). A short vibration reading is small; an image is big.
- **Complexity**: how much work each byte needs, in "CPU cycles per byte". Just averaging numbers is easy (60 cycles/byte); finding anomalies or preprocessing an image is hard (1,800–3,000 cycles/byte).

The total work is simply **size × complexity** = number of CPU cycles needed.

Task type	Difficulty (cycles/byte)	Typical size	In plain words
Sensor Aggregation	60	40 KB	add up and average readings: tiny job
Signal Filtering	250	80 KB	remove noise from a signal
FFT Feature Extract	800	160 KB	find which frequencies are in a vibration
Anomaly Detection	1,800	160 KB	decide whether something looks abnormal
Image Preprocessing	3,000	300 KB	clean and resize a camera image: heavy job

■ 4.2 Running it LOCAL on the ESP32

The ESP32 processor does **240 million cycles per second** (240 MHz), but part of that is already busy with its own background work (about 12 %). So:

- **Local time** = cycles needed ÷ cycles available per second.
- **Local energy** = processor power × local time. A busier processor draws slightly more power.

Example: an FFT task of 160 KB needs 160 × 1024 × 800 ≈ 131 million cycles. At about 211 million usable cycles per second that takes about **0.62 s** and uses about **79 millijoules** of energy.

Every task also has a **deadline of 5 seconds**: the answer must be ready within 5 s. A very heavy image task done locally can miss it.

5 WiFi: sending the task away

Offloading means sending the data over WiFi to the server. This is where things get interesting, because the quality of WiFi changes all the time. In our story the device sits on moving equipment, so its distance to the router keeps changing.

Everyday analogy
WiFi is like **talking to someone across a noisy room**.
• If you are close, you hear each other clearly and can talk fast.
• If you are far away, the voice is weak. You must talk slowly and clearly, and sometimes say things again.
• If many other people are talking (other WiFi devices), you must wait for gaps.
• Talking (transmitting) tires you more than listening.

Term	What it really is	In the noisy-room analogy
RSSI (dBm)	received signal strength. −40 is excellent, −70 is OK, −90 is very weak	how loud the other person sounds
Distance → RSSI	signal gets weaker with distance (and walls/obstacles)	further away = quieter voice
MCS (rate level)	WiFi picks a speed: MCS7 = fastest (65 Mbit/s raw), MCS0 = slowest (6.5)	how fast you dare to speak
PER	packet error rate: share of data packets that arrive broken and must be re-sent	how often you are misheard
Retransmissions	broken packets are sent again (up to 7 tries)	saying the sentence again
Transmit energy	the radio uses 0.60 W while sending and 0.33 W while waiting for the reply	talking tires you more than listening

Term	What it really is	In the noisy-room analogy
MQTT	a light "post office" protocol: the device publishes a message, a broker delivers it	dropping a letter in a postbox

The chain to remember: further away → weaker RSSI → slower speed (lower MCS) and more errors → more re-sending → **longer sending time** → **more radio energy**.

Real numbers for the same 160 KB FFT task: with a good signal (−62 dBm) the data goes at about 16 Mbit/s and sending takes about **85 ms**. With a weak signal (−86 dBm) the rate falls to about 4.3 Mbit/s, about 10 % of packets fail, and sending takes about **313 ms**, almost 4 times longer and 3 times more energy.

6 The edge server

The **edge server** is a powerful computer near the devices (in the same building or campus). It does **3 billion cycles per second**, about 12× faster than the ESP32. But it is **shared**: other users send it work too, more during office hours, and sometimes there are sudden bursts of traffic.

Everyday analogy
A **coffee shop** with one barista. When the shop is empty you get served immediately. When it is 50 % busy you wait a bit. When it is 90 % busy the queue explodes and you wait a long time. In maths this is called an **M/M/1 queue**: the waiting time grows like 1 ÷ (1 − busyness).

So for the same task: server 30 % busy → processing time × 1.4; server 90 % busy → × 10. While the device waits for the answer, its radio is listening, and listening also costs energy.

Remember this
OFFLOAD time = sending time (depends on WiFi) + server time (depends on how busy it is) + getting the answer back. OFFLOAD energy = radio sending + radio listening while waiting.

7 The decision: comparing two "bills"

For every task the twin calculates **both** options completely (time and energy for LOCAL, time and energy for OFFLOAD) and then gives each option a **bill**, called the **cost J**. The option with the **smaller bill wins**. There is no fixed rule like "offload when it is sunny"; the choice comes out of the numbers.

7.1 What goes into the bill

Bill item	Why it is there	Applies to
Energy × battery price λ	using energy is bad, and worse when the battery is low	both
Time ÷ deadline	slower answers are worse (measured as a fraction of the 5 s deadline)	both
+2 if the deadline is missed	a late answer is heavily punished	both
0.3 × packet error rate	a bad link is risky (the answer might not come back)	OFFLOAD only
0.3 × (server busyness)²	a crowded server is risky	OFFLOAD only

7.2 The battery price λ: why energy gets "expensive"

Everyday analogy
When you have lots of money, spending ₹10 does not worry you. When you are almost broke, the same ₹10 feels huge. The battery price λ (lambda) works the same way: it multiplies the energy part of the bill and **rises as the battery empties**.

Battery SOC	100 %	80 %	50 %	30 %	20 %	10 %	0 %
Price λ	1.00	1.36	2.41	3.34	3.86	4.42	5.00

7.3 A worked example: the same task in four situations

Task: FFT, 160 KB. Server 30 % busy. LOCAL always takes 621 ms and 79 mJ. Only the battery and the WiFi signal change:

Battery	Signal	OFFLOAD time / energy	Bill LOCAL	Bill OFFLOAD	Winner
80 %	good (−62 dBm)	152 ms / 73 mJ	0.232	**0.157**	**OFFLOAD**
80 %	weak (−86 dBm)	382 ms / 211 mJ	**0.232**	0.420	**LOCAL**
20 %	good (−62 dBm)	152 ms / 73 mJ	0.430	**0.340**	**OFFLOAD**
20 %	weak (−86 dBm)	382 ms / 211 mJ	**0.430**	0.948	**LOCAL**

What to notice:

- A good signal makes offloading cheap (fast and low energy), so OFFLOAD wins.
- A weak signal makes sending so slow that the radio burns 211 mJ, more than the 79 mJ of doing it locally, so LOCAL wins.
- At 20 % battery the price λ is 3.86 instead of 1.36. Every energy difference becomes about 3× more important. With a weak signal the gap grows from 0.19 to 0.52: **a low battery makes a wrong choice much more costly**.
- Rule of thumb: **low battery + good link → offloading saves battery; low battery + bad link → stay local.**

8 One real minute, step by step

Here is minute 754 from the dataset (Day 1, 12:34 pm), exactly as the twin computed it. Every minute follows these same steps.

1 · The clock moves	It is Day 1, 12:34 pm. The weather today is Partly Cloudy.
2 · The sun	Clouds let 394 W/m² through. The panel gives **0.335 W**.
3 · A task arrives	An **FFT Feature Extract** task with **105 KB** of data, complexity 775 cycles/byte → 83 million cycles.
4 · Option LOCAL	The ESP32 would need **385 ms** and **46 mJ**.
5 · The WiFi	The device is 19.9 m from the router; signal −71.6 dBm (OK). WiFi chooses MCS7; 7.5 % of packets fail. Sending takes 81 ms.
6 · The server	The server is **69 % busy** (lunch-time traffic), so processing takes 90 ms.

7 · Option OFFLOAD	Total **179 ms** (faster than local!) but **81 mJ**, because the radio sends and then waits for the busy server.
8 · Battery price	The battery is at 27.6 %, so energy is "expensive": $\lambda = 3.46$.
9 · The bills	LOCAL = $3.46 \times 0.046 + 0.385/5 =$ **0.237**. OFFLOAD = $3.46 \times 0.081 + 0.179/5 + 0.3 \times 0.075 + 0.3 \times 0.69^2 =$ **0.483**.
10 · Decision	**LOCAL** wins. Offloading would be faster, but it costs almost twice the energy, and with a low battery energy matters more than speed. The busy server adds risk too.
11 · Battery update	The sun gives 0.335 W, the device uses 0.141 W, so the battery **charges** by 0.026 % this minute.
12 · Record	All of this is saved as one row of the dataset, and the next minute begins with the new battery level.

9 The closed loop: today's choice changes tomorrow

The energy of the **chosen** option is really taken from the battery. So a choice made now changes the battery level, which changes the price λ, which changes the next decisions. This feedback is called a **closed loop**.

Everyday analogy
Spending money today leaves less for tomorrow, and having less tomorrow changes how you spend then. A bad habit repeated every minute adds up over a week.

That is why, later, every controller is tested by letting it **run the battery for a whole week**. A controller cannot just be right on average; its mistakes accumulate.

Remember this
Each minute: environment changes → task arrives → both options calculated → bills compared → cheaper one chosen → battery updated → next minute.

10 Why do we need AI at all?

Look back at step 7 of minute 754: to compute the OFFLOAD bill, the twin used the **exact** sending time, the **exact** packet error rate and the **exact current** server load. A simulator knows all of this. **A real device does not.** Before it sends the data, it cannot know how long sending will take or how busy the server is right now.

Everyday analogy
The twin's exact decision is like a student who has the **answer key**. Their answers are perfect, but in a real exam there is no answer key. We call this perfect decision-maker the **oracle**. We cannot use it on a real device, but it is the perfect teacher and referee: we can train other methods to copy it and score them against it.

What a real device **does** know at the moment it decides:

- its own battery level (SOC);
- the task: its size and type;

- how long and how much energy the task would take **locally** (its own processor is predictable);
- the WiFi signal strength measured **last minute**, and the server load reported **with the last answer**.

So the challenge is: **make nearly the oracle's decision using only this partial, slightly old information.** Three methods try this: Fuzzy only, ANN only, and the combination ANN + ANFIS.

11 ANN: a neural network that learns to predict

11.1 What is a neural network?

An **Artificial Neural Network (ANN)** is a mathematical function that **learns from examples** instead of being programmed with formulas. It is made of many simple units called **neurons**.

One neuron does three things:

- multiplies each input by a **weight** (how important that input is),
- adds everything up, plus a **bias** (a starting offset),
- squashes the result through an **activation function** (we use "tanh", which keeps values between −1 and +1, so the network can learn curves, not just straight lines).

```
Tiny made-up example: inputs task size = 0.8, signal = −0.3
weights 1.5, 2.0 bias = 0.1
sum = 0.8×1.5 + (−0.3)×2.0 + 0.1 = 0.7 output = tanh(0.7) = 0.60
```

Neurons are arranged in **layers**. The first layer looks at the raw inputs, the next layer combines the first layer's outputs into more complex patterns, and the last layer gives the answer. Our predictor has **6 inputs → 24 neurons → 16 neurons → 2 outputs**.

11.2 How does it learn?

> **Everyday analogy**
> Learning to throw a ball into a basket: you throw, see how far you missed, adjust your arm a little, and throw again. After thousands of throws you are accurate. A neural network does the same with its weights.

- **Guess**: feed an example in and get a prediction.
- **Measure the mistake**: compare with the true answer (the "loss").
- **Find who is to blame**: back-propagation works out how much each weight contributed to the mistake.
- **Adjust**: nudge every weight a little in the direction that reduces the mistake. We use the **Adam** method, which adapts the step size automatically.
- **Repeat** for all 8,000 training examples, 60 times over (60 "epochs").

11.3 What our ANN learns

Input (known): task size, task complexity, last WiFi signal, last server load, time of day.
Output (unknown, to predict): how long offloading will take and how much energy it will cost.

Result: on days it never saw during training, its predictions are typically within about **5 %** of the truth. For minute 754 it predicted **180 ms** (real 179 ms) and **83.6 mJ** (real 81.1 mJ).

11.4 Practice exam and real exam

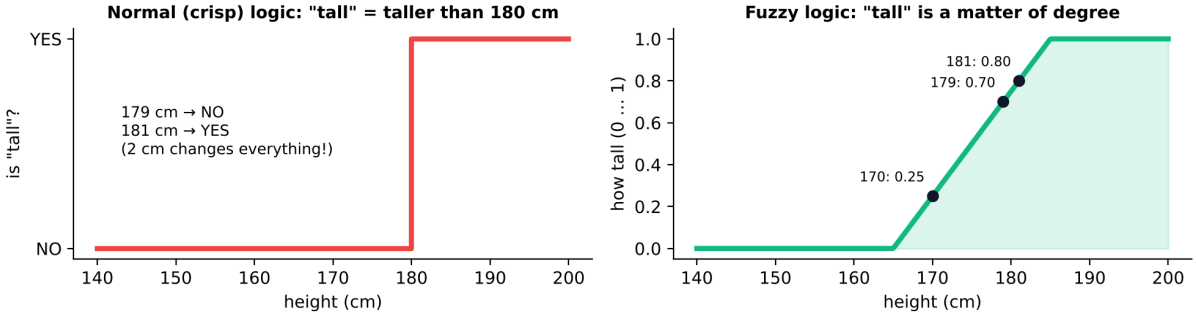
We never test a student on the exact questions they practised. In the same way, the ANN is **trained** on the first 80 % of the data and **checked** on the last 20 %, which it has never seen (the "validation" data). If it only memorised the training data, it would fail the check. This is called **overfitting**.

> **Remember this**
> ANN = a learnable function. Show it many examples, it adjusts its weights until its guesses are right. Strength: accurate predictions. Weakness: it cannot explain **why** (a "black box").

12 Fuzzy logic: reasoning with words like humans do

12.1 The problem with yes/no logic

Normal computer logic is **crisp**: something is either true (1) or false (0). If we say "a person is tall when taller than 180 cm", then 179 cm is "not tall" and 181 cm is "tall". A 2 cm difference flips everything, which is not how humans think.



Crisp logic jumps from NO to YES. Fuzzy logic rises smoothly: 170 cm is "a bit tall" (0.25), 181 cm is "quite tall" (0.80).

Fuzzy logic lets something be **partly true**, with a degree between 0 and 1. The curve that gives the degree is called a **membership function**. Each word ("Low", "Medium", "High") has its own curve, and the curves overlap, so a value can be partly "Low" and partly "Medium" at the same time.

12.2 The four steps of a fuzzy controller

Step	What happens	Everyday version
1 Fuzzify	turn each number into degrees for each word	"25 % battery is mostly Low (0.67) and a bit Medium (0.25)"
2 Apply rules	IF ... AND ... THEN ... rules; AND takes the **smallest** degree	"a chain is only as strong as its weakest link"
3 Combine	every rule votes for LOCAL, MAYBE or OFFLOAD with the strength it got	collect all the votes
4 Defuzzify	turn the votes into one number between 0 and 1 (the balance point)	find where a cardboard cut-out of the shape would balance on a finger

The final number is the **offload score**: above 0.5 means OFFLOAD, below means LOCAL.

12.3 Our "Fuzzy only" controller

It looks at 4 things: **Battery** (Low / Medium / High), **Task load** = local time (Light / Medium / Heavy), **WiFi link** = last RSSI (Poor / Fair / Good) and **Server load** (Light / Busy / Overloaded). An "expert" (us) wrote 12 rules, for example:

- IF Task load is Light → LOCAL (sending a tiny job costs more than doing it)
- IF Task load is Medium AND Server load is Busy → LOCAL (waiting for the server costs energy)
- IF Task load is Heavy AND WiFi link is Good → OFFLOAD (big job, easy to send)
- IF Task load is Heavy AND Battery is Low → OFFLOAD (doing it locally would drain the battery)

12.4 Minute 754 through fuzzy logic

- **Fuzzify:** Task load 385 ms → Medium = 1.0. Server load 68.7 % → Busy = 0.32 and Overloaded = 0.43 (it is between the two words).
- **Rules that fire:** "Medium task AND Overloaded server → LOCAL" with strength min(1.0, 0.43) = **0.43**; "Medium task AND Busy server → LOCAL" with strength min(1.0, 0.32) = **0.32**.
- **Combine + defuzzify:** only LOCAL received votes → score **0.20** → **LOCAL**. Correct, the same as the oracle.

Remember this
Fuzzy logic = rules written in human words, with "partly true" degrees. Strength: anyone can read why it decided. Weakness: a human had to guess where "Busy" starts and ends, and it cannot know the real offload cost.

13 ANFIS: fuzzy rules that learn by themselves

13.1 The idea

Fuzzy logic is understandable, but someone must design the word-curves and the rules by hand. Neural networks learn automatically, but nobody can read them. **ANFIS** (Adaptive Neuro-Fuzzy Inference System) combines the two: it is a fuzzy rule system **built like a neural network**, so it can be **trained from examples** but still reads as IF–THEN rules.

Everyday analogy
A new employee writes their own rule book ("if the customer is angry and the order is late, give a discount"). An experienced manager (the oracle) shows them thousands of real cases with the correct action. The employee keeps adjusting what "angry" and "late" mean and what each rule should do, until their rule book matches the manager. The rule book stays readable the whole time.

13.2 Our ANFIS

It looks at 4 things, and two of them come from the ANN's predictions:

Input	Words	Meaning
Battery SOC	Low · Medium · High	how full the battery is
Energy ratio	Offload cheaper · Similar · Local cheaper	predicted offload energy compared with local energy (from the ANN)
Time gain	Small · Large	how much faster offloading is predicted to be (from the ANN)

Input	Words	Meaning
Server load	Light · Busy	last reported server load

Every combination of words is one rule: 3 × 3 × 2 × 2 = **36 rules**. Each rule outputs an offload score.

13.3 The 5 layers in plain words

Layer	What it does	Plain words
1	membership degrees (bell-shaped curves)	"How Low / Medium / High is the battery right now?"
2	multiply the degrees inside each rule	"How well does this situation match each rule?"
3	divide by the total	"What share of the decision does each rule get?" (shares add up to 100 %)
4	each rule computes its own score with a small formula	"What does each rule recommend here?"
5	weighted average of the rule scores	"Final answer: the offload score"

13.4 How it learns (two moves, repeated 30 times)

- **Move 1, fix the rule outputs:** with the word-curves fixed, the best formula for every rule can be calculated exactly in one go (a method called least squares).
- **Move 2, adjust the word-curves:** with the rule outputs fixed, the centre and width of every bell curve are nudged a little to reduce the mistakes (gradient descent, the same idea as the ANN).
- The teacher is the oracle: for each training minute it says how strongly OFFLOAD is better (1), LOCAL is better (0) or it is a tie (0.5).

13.5 Minute 754 through ANFIS

- Inputs: battery 27.6 %, energy ratio "Similar" (the ANN predicts offloading costs about 1.8× the local energy), time gain "Small" (only 4 % of the deadline), server "Busy" (68.7 %).
- Strongest rule: **"SOC is Medium · Energy is Similar · Time gain is Small · Server is Busy"**. It gets 50 % of the decision and recommends a score of 0.03 (strongly LOCAL).
- Final score **0.16 → LOCAL**. Correct, and we can say exactly which rule caused it.

14 The hybrid: ANN + ANFIS working together

Each method alone has a weakness:

Method	Good at	Weak at
Fuzzy only	explaining its decisions in words	it guesses thresholds and does not know the real offload cost
ANN only	accurate decisions	it cannot explain anything (a black box)
ANN + ANFIS	**accurate AND explainable**	a little more complex to build

How they work together: Predict → Reason → Decide.

- **Predict (ANN):** "If I offload this task, it will take about 180 ms and about 84 mJ."
- **Reason (ANFIS):** "Battery is medium-low, offloading costs more energy for little time gain, and the server is busy. My learned rule says: stay local."

- **Decide:** score 0.16 → LOCAL.

> **Everyday analogy**
> Deciding whether to take an umbrella. The **weather forecast** (ANN) predicts "70 % chance of rain". Then your **common-sense rules** (fuzzy) decide: "IF rain is likely AND I am walking far → take the umbrella". The forecast gives knowledge; the rules turn it into an explainable action. ANFIS also learned the rules from experience.

One clever detail: ANFIS is trained on the ANN's **predictions**, not the true values. In real use it will only ever receive predictions, so it learns to work well even when the ANN is slightly wrong.

15 How we test fairly

- **Practice vs exam:** models learn from the first 80 % of the 7-day dataset and are checked on the last 20 %, which they never saw.
- **A brand-new week:** then the twin generates a completely new week (different random weather, distances and traffic).
- **Each controller drives the battery itself** for that whole week (closed loop), so mistakes accumulate as they would on a real device.
- **The oracle is the referee:** for every minute we know what the perfect choice would have been.

Score	Question it answers	Better is
Agreement with oracle	How often did it make the perfect choice?	higher
Task energy	How much battery did all the tasks use in the week?	lower
Deadline misses	How many answers were later than 5 seconds?	lower
Regret	On average, how much bigger was its bill than the best possible bill?	lower (0 = perfect)

16 Results in plain words

Controller	Perfect choices	Energy used	Late answers	Can explain itself?
Oracle (answer key)	100 %	911 J	0	— (not possible on a real device)
Fuzzy only	92.7 %	978 J	16	yes
ANN only	95.9 %	951 J	11	no
ANN + ANFIS	**96.6 %**	**932 J**	**9**	**yes**

- The **hybrid comes closest to the perfect answer key**, uses the least energy and is late the fewest times among the realistic methods.
- It is also the only accurate method that can **explain each decision** with readable rules.
- Fuzzy-only is understandable but makes the most mistakes, because its thresholds were guessed by hand.
- ANN-only is accurate, but nobody can see why it decides what it decides.
- The battery curves of all four look almost the same, because a task uses little energy compared with the 0.14 W the device always needs. The differences show up in total task energy and late answers.

17 Common confusions, cleared up

"Does sunshine directly decide LOCAL or OFFLOAD?"

No. The sun only charges the battery. The battery level changes the energy price λ, and that affects the decision. Solar works through the battery.

"Isn't offloading always better because the server is faster?"

No. Sending data costs radio energy, a weak signal means re-sending, and a busy server means waiting. For small tasks or bad links, LOCAL is cheaper.

"Is low battery always a reason to offload?"

Only if the link is good. With a bad link, offloading uses more energy than computing locally, so a low battery then favours LOCAL.

"Why not just use the oracle on the real device?"

Because it needs future, exact information (true sending time, current server load) that a real device does not have before sending.

"What is the difference between fuzzy logic and probability?"

Probability says how likely something is. Fuzzy says how true a vague word is. "Battery is 0.67 Low" is not a 67 % chance; it means the battery is fairly low.

"Is the data real?"

It is simulated from physical equations with realistic parameters, and 20 automatic checks confirm it behaves physically (e.g. further away → weaker signal → slower).

"Why write the ANN and ANFIS ourselves instead of using a library?"

To show exactly how they work, and so that every step (neurons, back-propagation, membership functions, rule learning) is visible in the code.

18 A 2-minute script to explain the project

Say it like this

"My project is a **digital twin** of a solar-powered IoT device, an ESP32 with a small solar panel and battery. Every minute it gets a processing task and must decide: do it **locally**, which is slow and uses the processor, or **offload** it over WiFi to an edge server, which is faster but costs radio energy and depends on the signal and how busy the server is.

The twin simulates the sun, battery, processor, WiFi and server with physical equations. For each task it computes the time and energy of both options, turns them into a cost where energy gets more expensive as the battery drops, and picks the cheaper one. I verified with 20 automatic checks that the physics behaves correctly.

But a real device cannot know the exact offload cost in advance. So I built an intelligent layer: a **neural network** predicts the offload time and energy from what the device knows, and an **ANFIS**, a fuzzy rule system that learns from data, turns those predictions into a decision with readable IF–THEN rules.

On a new, unseen week, this hybrid matched the ideal decision **96.6 %** of the time. It used less energy and missed fewer deadlines than fuzzy logic alone or a neural network alone, and it can explain every decision. Everything is shown live in a Streamlit dashboard with a 3D view of the system."

19 Test yourself

Try to answer before reading the answer.

1. What does SOC mean, and why does a low SOC make energy "expensive"?

Answer: State of charge (how full the battery is). With little energy left, wasting it risks the device dying, so the price λ rises from 1 (full) to 5 (empty).

2. The signal drops from −62 to −86 dBm. What happens to offloading and why?

Answer: WiFi switches to a slower rate and more packets fail and are re-sent, so sending takes longer and uses more radio energy. LOCAL becomes the better choice.

3. Why did minute 754 choose LOCAL even though OFFLOAD was faster?

Answer: Offloading needed 81 mJ against 46 mJ locally (the busy server made the radio wait), and at 27.6 % battery energy is expensive (λ = 3.46). The bill was 0.237 for LOCAL against 0.483 for OFFLOAD.

4. What does the ANN predict, and from what?

Answer: Offload time and offload energy, from task size, complexity, last signal strength, last server load and time of day.

5. In fuzzy logic, what does "AND" do?

Answer: It takes the smallest of the degrees (the weakest link).

6. What does ANFIS learn that a hand-written fuzzy system does not?

Answer: The shapes of the word-curves (where "Busy" starts, for example) and what each rule should output, both learned from the oracle's examples.

7. Why test on a new week with each controller driving the battery?

Answer: To check that the methods work on situations they never saw, and that their mistakes do not pile up over time.